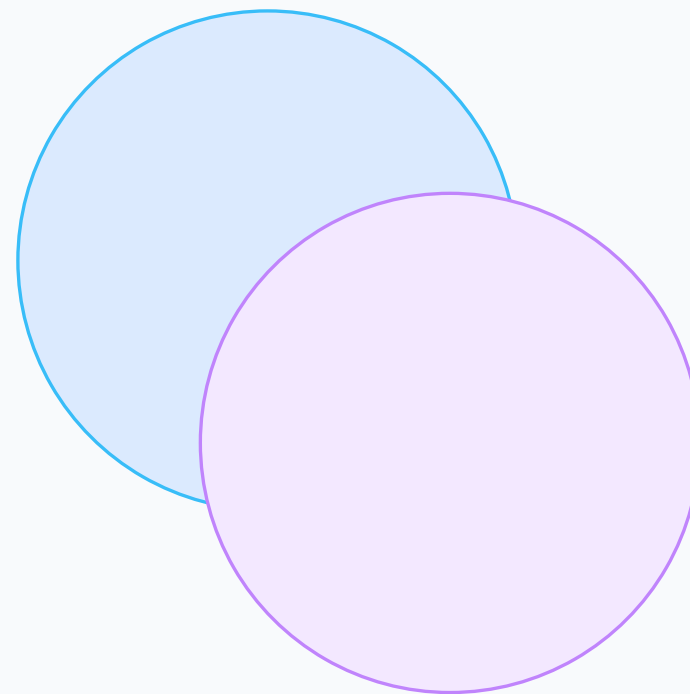


Асинхронная загрузка без зависаний

Главный поток, отмена, исключения, `job_pool` и FPS

**400 МБ ресурсов
не должны остановить игру**

Цель: загрузить в фоне, уметь отменить, не потерять инварианты.



Какие вопросы закрываем

После презентации должна сложиться полная картина

Главный поток

почему нельзя долго
работать

`tb::job_pool`

почему не каждая очередь
задач подходит

Отмена

как остановить загрузку
культурно

Тайминги

16 мс, 0.8 с, 2 с, 5 с ANR

Потоки

почему нельзя просто убить
thread

Callbacks

почему try/catch
обязателен

FPS

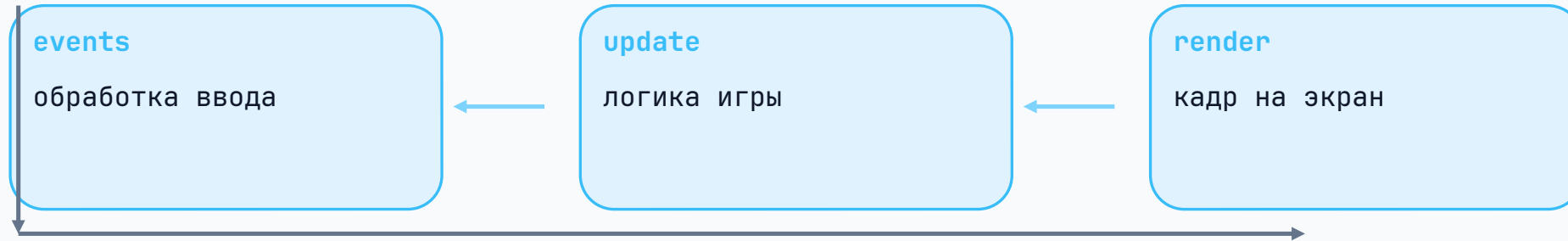
как зависимости задач
упираются в кадры

Result

как вернуть данные и
ошибки

Главный поток: сердце приложения

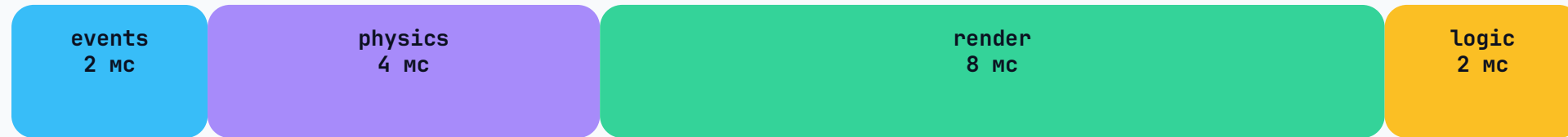
Он качает события, обновление и рендер



Если один блок завис, игрок видит не «работу», а мертвое окно.

Бюджет кадра: всего 16 мс

60 FPS = 1 / 60 секунды



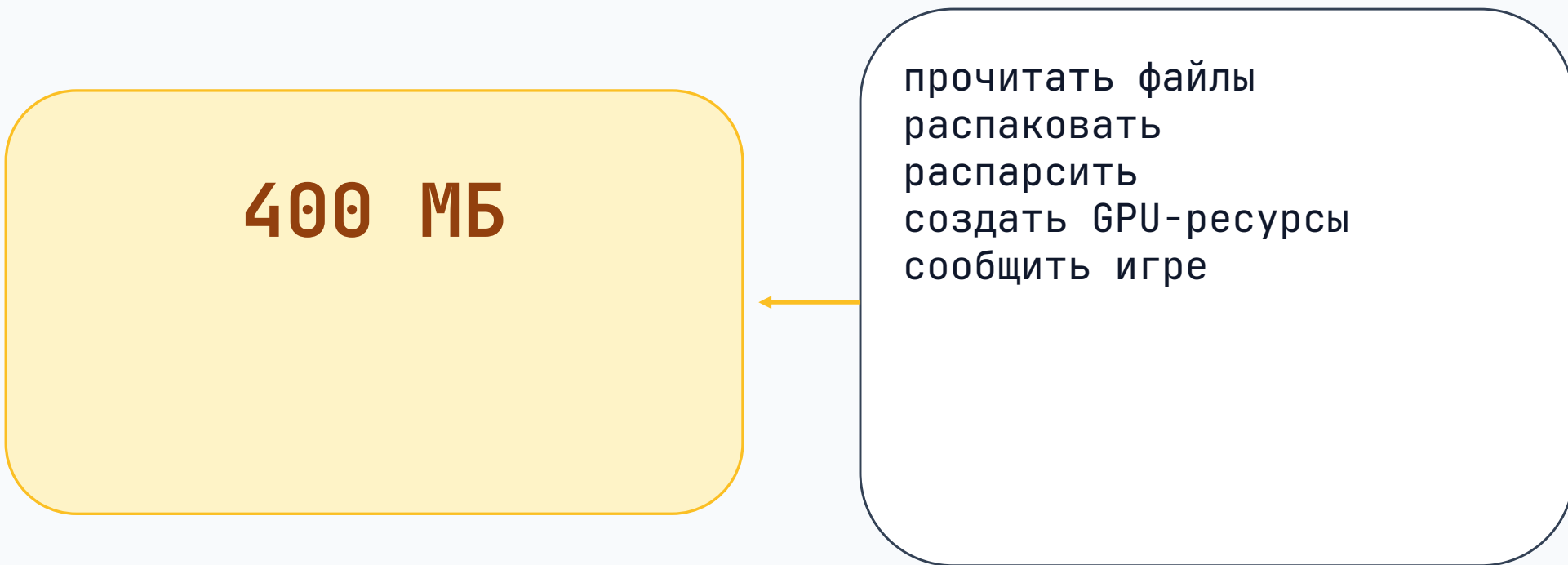
Любой `wait()`, `sleep()`, `join()` или синхронный I/O крадет этот бюджет.

Загрузка уровня: 400 МБ

На диске это данные, для игрока это ожидание

400 МБ

прочитать файлы
распаковать
распарсить
создать GPU-ресурсы
сообщить игре



```
graph LR; A[прочитать файлы  
распаковать  
распарсить  
создать GPU-ресурсы  
сообщить игре] --> B[400 МБ];
```

Почему нельзя долго работать на main

Потому что main отвечает за ощущение жизни

События не читаются

ОС считает окно зависшим

Кадры не рисуются

игрок видит фриз

Ввод не работает

кнопка «назад» бесполезна

Закрытие тормозит

можно получить ANR

Тайминги, которые держим в голове

Не точные законы природы, а инженерные ориентиры

16мс

кадр при 60 FPS

0.8с

заметная
реакция

2с

ожидание
нервирует

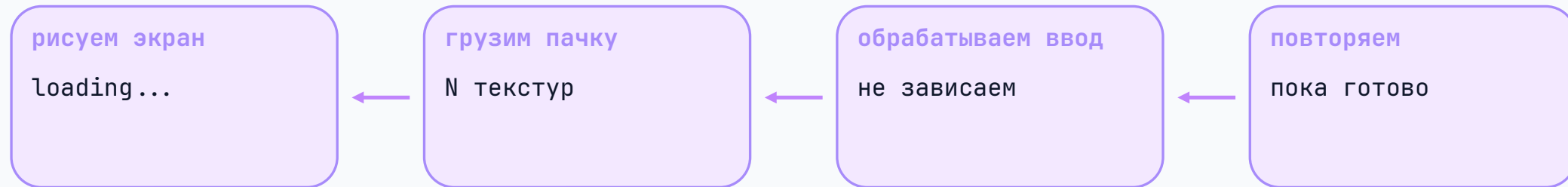
5с

верхняя зона
ANR

Практическое правило: целевой лимит делим на 2 и проверяем отмену чаще.

Старый способ: кооперативная многозадачность

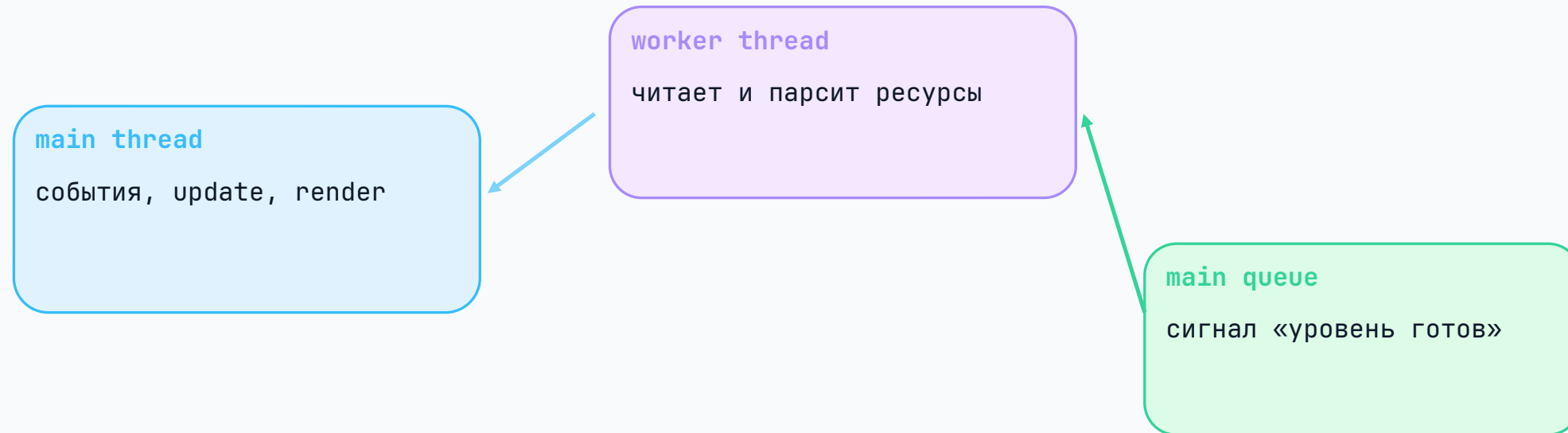
Одно ядро, один поток, много добровольных уступок



Современная отмена устроена похоже: поток сам регулярно проверяет «продолжать?»

Современный путь: отдельный поток

Main остается отзывчивым, worker делает долгую работу



Почему «просто поток» быстро усложняется

Появляются жизненный цикл, ошибки и синхронизация

экран закрыт

объекты уже уничтожены

файл не прочитан

исключение ушло в thread

вечный цикл

готовность не придет

mutex захвачен

join или kill ломает всех

render ждет texture

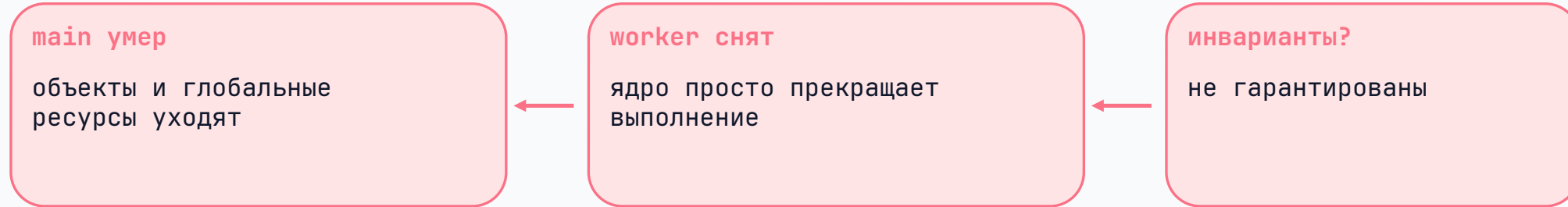
main ждет закрытие

очередь main

мелкие задачи стали 60/сек

Если `main` завершился раньше `worker`

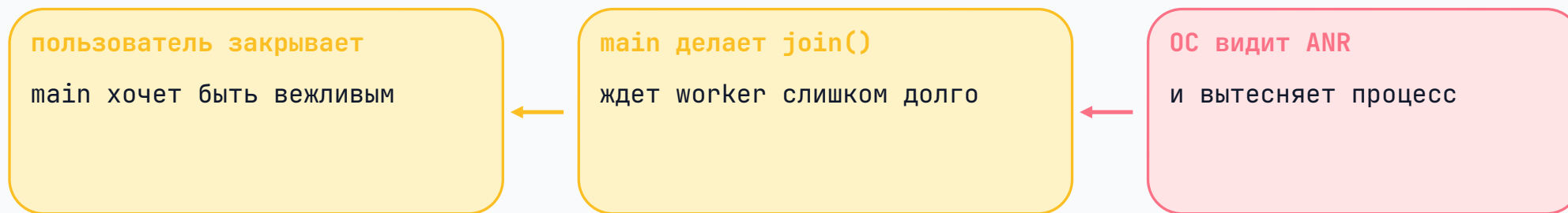
ОС не делает красивую отмену вашего кода



Файл кэша, лог, mutex, OpenGL context: всё может остаться в промежуточном состоянии.

join() тоже может быть проблемой

Корректность не должна превращаться в зависание при выходе



Отмена должна быть запрошена заранее и быстро доходить до `interruption point`.

Почему нельзя убивать поток через API ОС

Thread kill обрывает код между двумя инструкциями

RAII не сработал

деструкторы не раскрутили
стек

mutex остался

синхронизация сломана

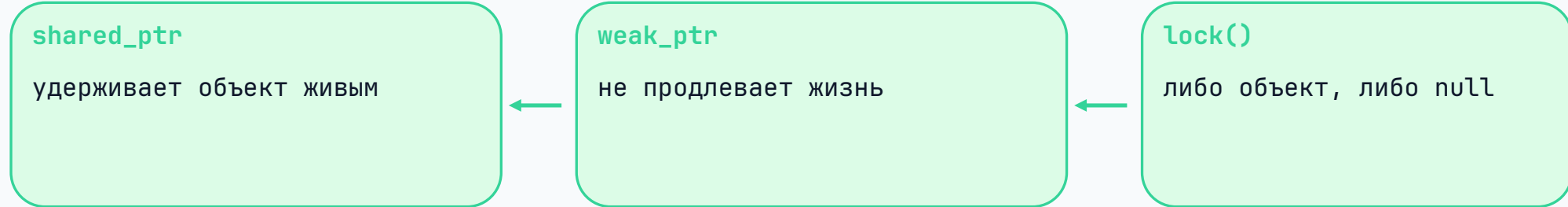
файл оборван

кэш потерял инвариант

Правильный путь: попросить остановиться, дойти до безопасной точки, выйти обычным return или исключением, которое поймано.

Владение объектами между потоками

Классический ответ: `shared_ptr` + `weak_ptr`



Глобальный объект не «вечный»: при завершении процесса он тоже разрушается.

std::jthread + stop_token

C++20 дает авто-join и кооперативную отмену

```
void do_work(std::stop_token stoken) {  
    for (int i = 0; i < 100; ++i) {  
        if (stoken.stop_requested()) {  
            return; // безопасная точка остановки  
        }  
  
        std::this_thread::sleep_for(100ms);  
        load_next_chunk();  
    }  
}  
  
std::jthread worker(do_work);
```

Отмена остается кооперативной

`stop_token` сам ничего не прерывает

часто проверяем

перед I/O, после пачки, в циклах

быстро выходим

`return` из безопасной точки

чистим RAII

стек раскручивается штатно

не блокируем `main`

`join` только с понятным лимитом

Boost: interruption points

Старый, но выразительный паттерн отмены

```
void do_work() {  
    try {  
        for (int i = 0; i < 100; ++i) {  
            boost::this_thread::interruption_point();  
  
            boost::this_thread::sleep_for(100ms);  
            load_next_chunk();  
        }  
    } catch (const boost::thread_interrupted&) {  
        // graceful shutdown  
    }  
}  
  
thread.interrupt();  
thread.join();
```

Почему try/catch обязателен в callback

Исключение в чужом потоке не вернется магически на main

callback бросил

нет файла, ошибка парсинга,
assert

исключение не поймано

std::terminate / падение
процесса

лог потерян

причину сложно найти

Граница потока должна ловить все, логировать, переводить ошибку в результат и уведомлять владельца.

Шаблон безопасной границы worker

Пользовательский код не должен валить весь процесс без лога

```
void worker_entry(std::function<void()> user_callback) {  
    try {  
        user_callback();  
        post_result(success{});  
    } catch (const std::exception& e) {  
        log_error(e.what());  
        post_result(failure{e.what()});  
    } catch (...) {  
        log_error("unknown worker exception");  
        post_result(failure{"unknown"});  
    }  
}
```

Где ставить точки отмены

Ответ рождается из таймингов

до длинного I/O

чтобы не начинать лишнее

после каждой пачки

текстура, mesh, chunk

в больших циклах

не реже целевого лимита

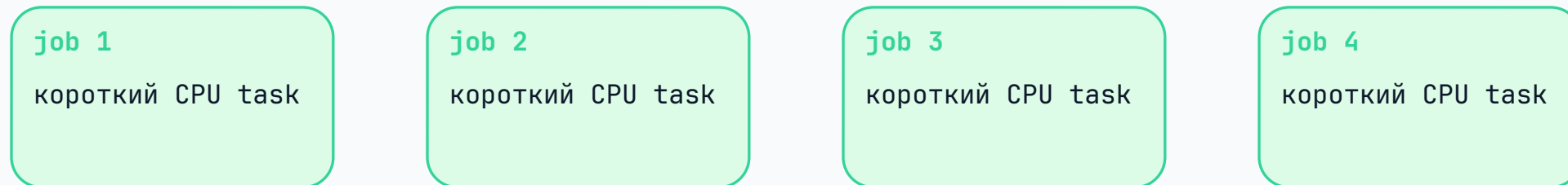
перед callback

владелец мог исчезнуть

Для закрытия без ANR: не зависать дольше ~2.5 секунд локально.

Что хорошего в `job_pool`

Он отлично утилизирует ядра для коротких независимых задач



Консольная утилита или batch processing: да, можно загрузить железо на максимум.

Проблема загрузки через `tb::job_pool`

В графическом приложении долгие задачи могут отравить пул

wait/sleep внутри job

воркер занят, новые задачи стоят

нет контроля отмены

чужой пул не знает ваш `stop_token`

исключения опасны

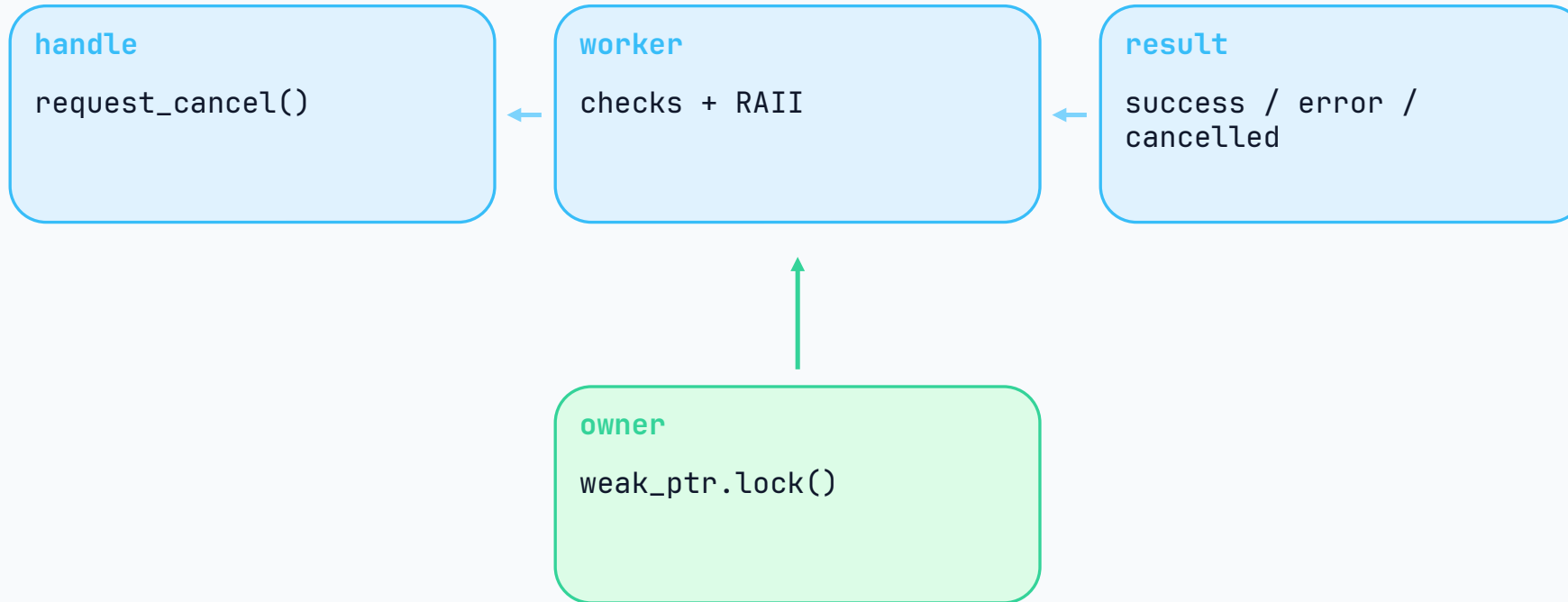
`callback` может убить процесс

I/O непредсказуем

`latency` ломает FPS-план

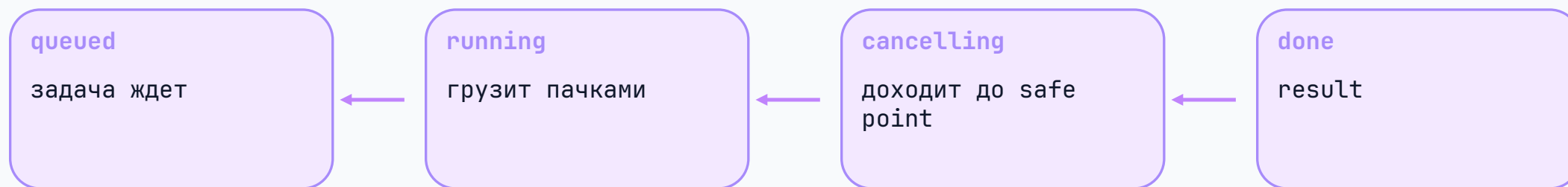
Нужна отдельная модель загрузки

Не «любой job», а управляемая async-операция



Состояния cancellable loading

Отмена - это штатный результат, а не авария



done может означать: success, failure или cancelled.

Main queue: только крупные события

Не привязывайте каждую мелкую загрузку к FPS

плохо

texture готова → main →
следующая texture

лучше

worker queue запускает
следующую сразу

main

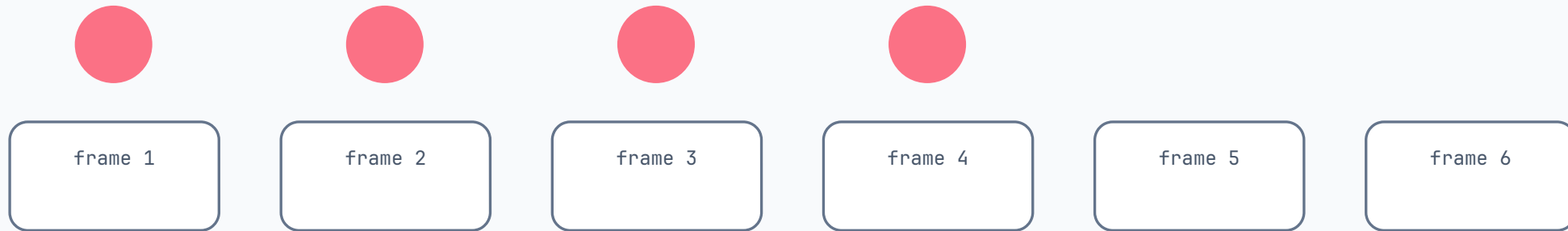
получает «уровень
готов»

Сигнал на main раз в кадр - нормально для крупных фаз, плохо для сотен мелких зависимых задач.

Как зависимость задач упирается в FPS

Если следующая загрузка стартует только из main queue

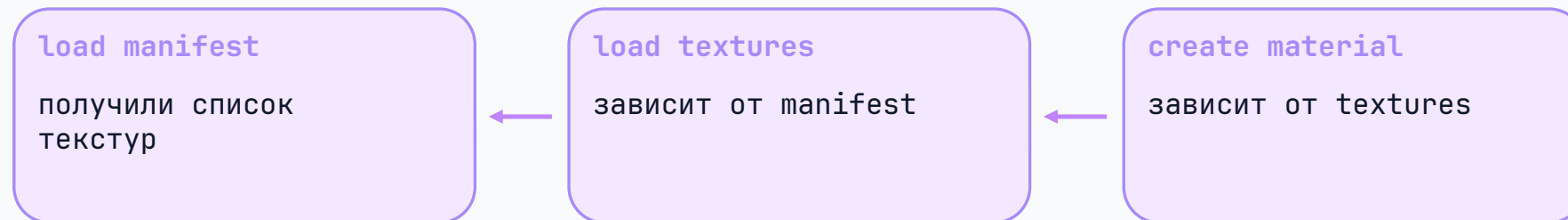
texture A готова → событие main → кадр → старт texture B



Итог: максимум около 60 таких переходов в секунду при 60 FPS.

Когда результат A влияет на задачу B

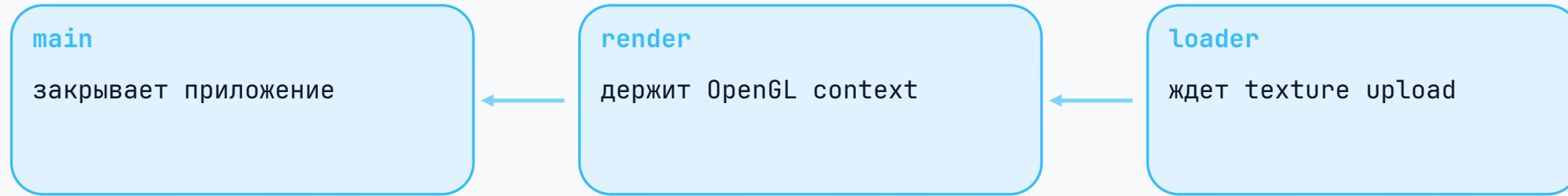
Опасность: граф зависимостей спрятан в callback'ax



Если каждый переход идет через main, граф загрузки начинает тикать с частотой игры.

Render thread и контекст графики

Еще одна причина не блокировать закрытие



Цикл ожиданий между потоками превращает «заккрыть приложение» в ANR.

Минимальный протокол результата

Main должен получить не только «готово»

success

ресурсы готовы

failure

ошибка с диагностикой

cancelled

владелец передумал

owner missing

weak_ptr не залочился

Как вернуть результат из worker

Есть несколько вариантов, у каждого своя цена

`std::future`

`get()`, `wait()`, `wait_for()`

`main queue`

callback на следующем кадре

`shared object`

данные под mutex

`typed result`

success / error / cancelled

Главное правило: `main` не должен блокироваться, пока ждет результат.

packaged_task + future

Стандартная библиотека умеет переносить значение и исключение

```
std::packaged_task<om::data(std::stop_token)> task(
    [](std::stop_token stoken) → om::data {
        if (stoken.stop_requested()) {
            return {};
        }

        if (something_bad()) {
            throw std::runtime_error("load failed");
        }

        return make_loaded_data();
    });

std::future<om::data> future = task.get_future();
std::jthread worker(std::move(task));
```

future: не блокируем main

Состояние можно проверить без ожидания

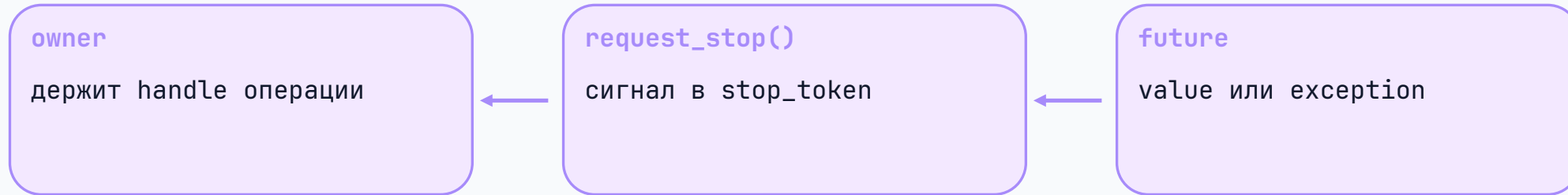
```
using namespace std::chrono_literals;

auto status = future.wait_for(0s);
if (status == std::future_status::ready) {
    try {
        om::data data = future.get();
        apply_loaded_data(std::move(data));
    } catch (const std::exception& e) {
        show_load_error(e.what());
    }
}

// future.get() перебросит исключение из worker.
```


Отмена + future

`request_stop()` просит `worker` завершиться, `result` все равно типизирован



Даже при исключении у вызывающего кода есть точка, где его можно поймать и превратить в UI-сообщение.

Альтернативы future

Они работают, но синхронизацию придется держать руками

объект + mutex

легко ошибиться с lock lifetime

lambda на main

результат придет только в кадре

сырой buffer

указатель + размер плохо типизированы

ручные ошибки

нужен единый error channel

Практический чек-лист

Что должно быть в реальной аsync-загрузке

- ✓ `main` никогда не делает долгий `wait/sleep/join`
- ✓ у операции есть `handle` и `request_cancel()`
- ✓ `worker` регулярно проверяет `stop/interruption`
- ✓ граница `worker` ловит все исключения
- ✓ результат типизирован: `success/failure/cancelled`
- ✓ владельцы передаются через `weak_ptr`
- ✓ мелкие зависимости не идут через `main queue`

Ментальная модель

Никакой серебряной пули нет

Асинхронность не отменяет кооперативность.

Потоки работают безопасно только когда вы явно описали владение, отмену, ошибки, очереди и тайминги.

Сначала инварианты, потом производительность.